Flutter Clean Architecture

Feature-First Developer Guidelines

Core principle: Business logic must remain independent from infrastructure and UI

1. Architecture Overview

This document describes the recommended Feature-First + Clean Architecture approach for scalable Flutter applications. The architecture enforces a strict dependency direction that keeps business logic isolated from framework and infrastructure concerns.

Dependency Rule:

Presentation → Domain ← Data

- Presentation depends on Domain
- Data depends on Domain
- Domain depends on nothing

The architecture is built on:

- Feature-First structure — code is organised by feature, not by technical layer
- Clean Architecture principles — strict layer separation with inverted dependencies
- SOLID principles — single responsibility, open/closed, Liskov, interface segregation, dependency inversion
- Infrastructure isolation — UI and business logic never know about API or persistence details

Calls Flow

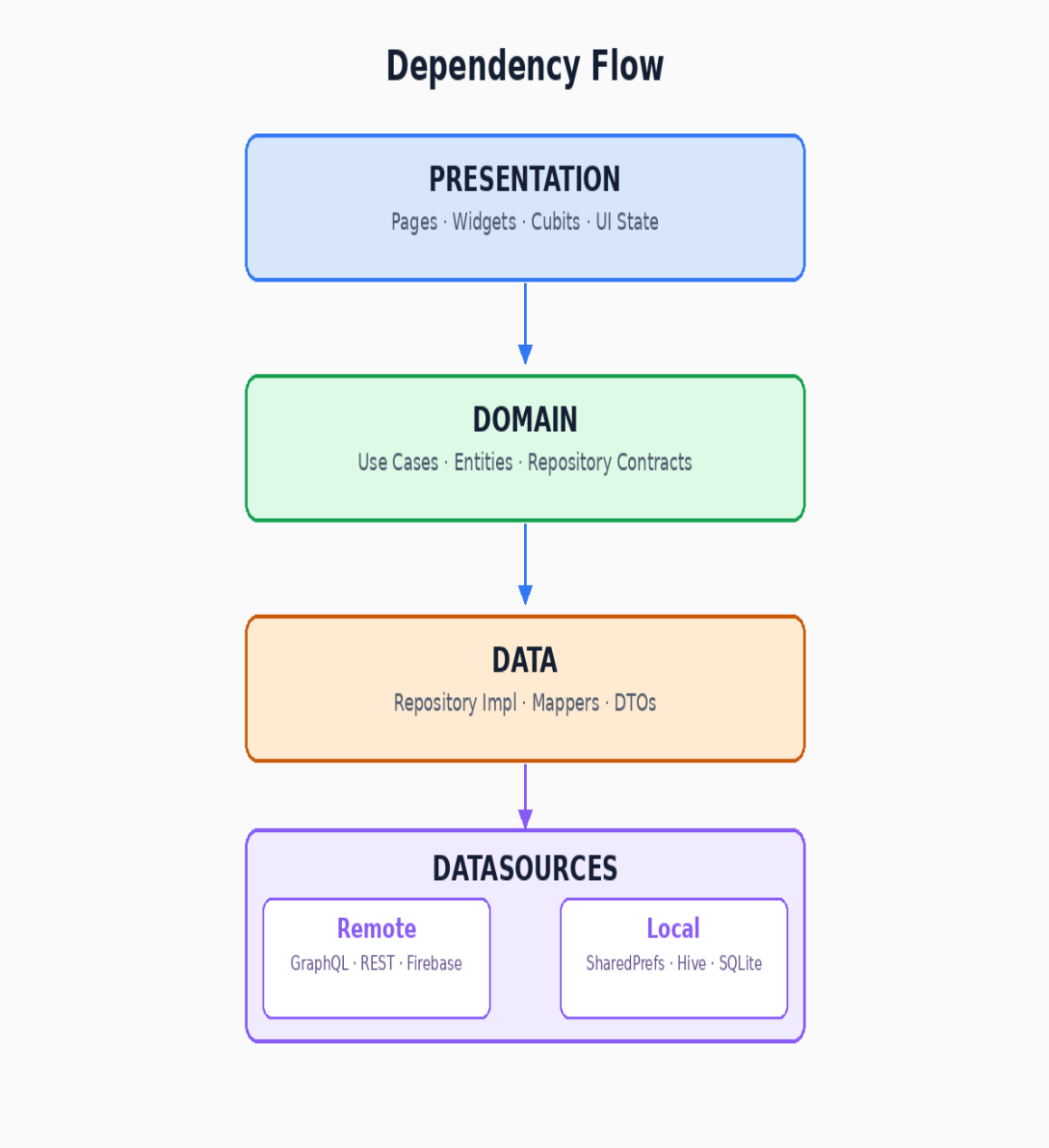


Figure 1 — Dependency flow across all architectural layers

2. Feature-First Structure

Each feature lives in its own directory with three layers: data, domain, and presentation. All feature code is co-located, making it easy to add, remove, or refactor entire features without affecting others.

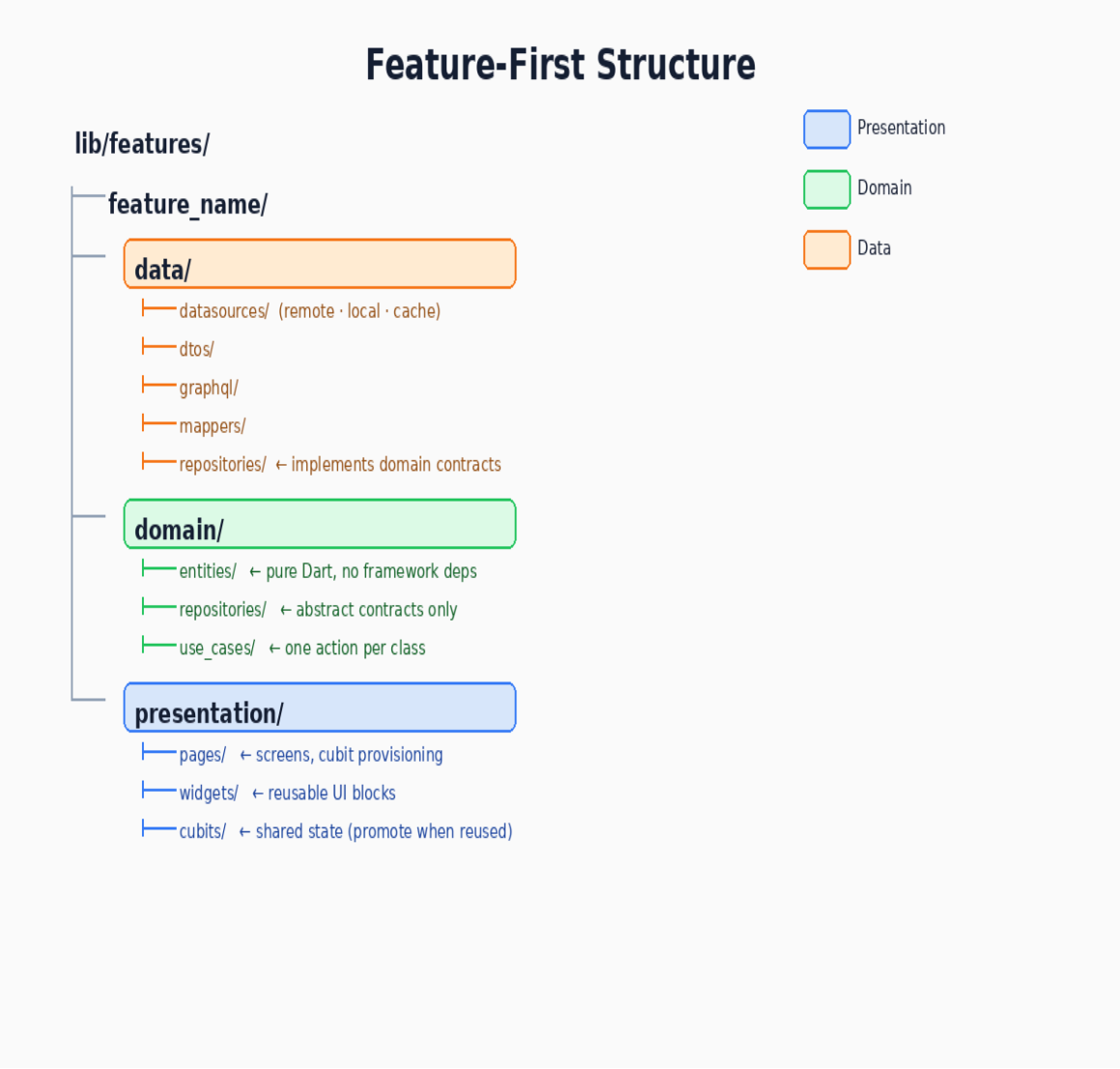


Figure 2 — Feature-first folder structure with layer colour coding

3. Presentation Layer

The presentation layer is responsible for UI rendering, user interaction, state management, and navigation orchestration. It contains pages, widgets, and cubits/blocs.

> Presentation must NOT contain: API calls, DTOs, persistence logic, or business logic.

Pages

Pages represent complete screens and route entry points. They are responsible for high-level UI orchestration, Cubit provisioning, and coordinating navigation flow.

Reusable widgets should preferably remain navigation-agnostic and communicate user interactions through callbacks or presentation events.

Widgets

Widgets represent reusable UI components or feature-specific UI blocks. They should receive already-prepared state or models, and must avoid business logic or repository access.

Cubits / State Management

Cubits are responsible for presentation state, UI orchestration, calling use cases, and exposing immutable state.

> Cubits must NOT: access datasources directly, know about DTOs, or contain infrastructure logic.

Presentation Structure Recommendation

Keep widgets and cubits close to the page that owns them. Only promote them to the shared level when reused across multiple pages.

```
presentation/
│
├── pages/
│   ├── products_page/
│   │   ├── products_page.dart
│   │   ├── cubit/
│   │   └── widgets/
│   └── details_page/
│       ├── details_page.dart
│       ├── cubit/
│       └── widgets/
│
├── widgets/            ← shared across pages
│   ├── shared_feature_widget.dart
│   └── reusable_feature_card.dart
```

```
│
└── cubits/              ← shared across pages
    └── feature_shared_cubit.dart
```

4. Domain Layer

The domain layer is the heart of the application. It contains business rules, entities, repository contracts, and use cases. It must remain framework-independent, infrastructure-independent, and written in pure Dart.

> Domain Rule: No Flutter imports. No API knowledge. No DTO knowledge.

Entities

Entities represent business models and domain concepts. They must not contain serialization logic and must not depend on APIs or frameworks.

Repository Contracts

Repositories define business-facing data access contracts. They expose entities only, remain source-agnostic, and hide all infrastructure details. They must not expose GraphQL, REST, DTOs, or cache strategies.

Use Cases

Use cases represent a single business action. They orchestrate business logic while remaining focused and avoiding infrastructure concerns.

Use Case	Semantic Meaning
`LoadProductsUseCase`	Source-agnostic retrieval. Repository-backed loading. Preferred default.
`FetchProductsUseCase`	Explicit remote/network retrieval. Bypass-cache operations.
`GetCachedProductsUseCase`	Synchronous, local, or in-memory retrieval.

5. Naming Conventions

Consistent naming communicates intent at a glance. The following legend and table summarise the recommended conventions across all layers.

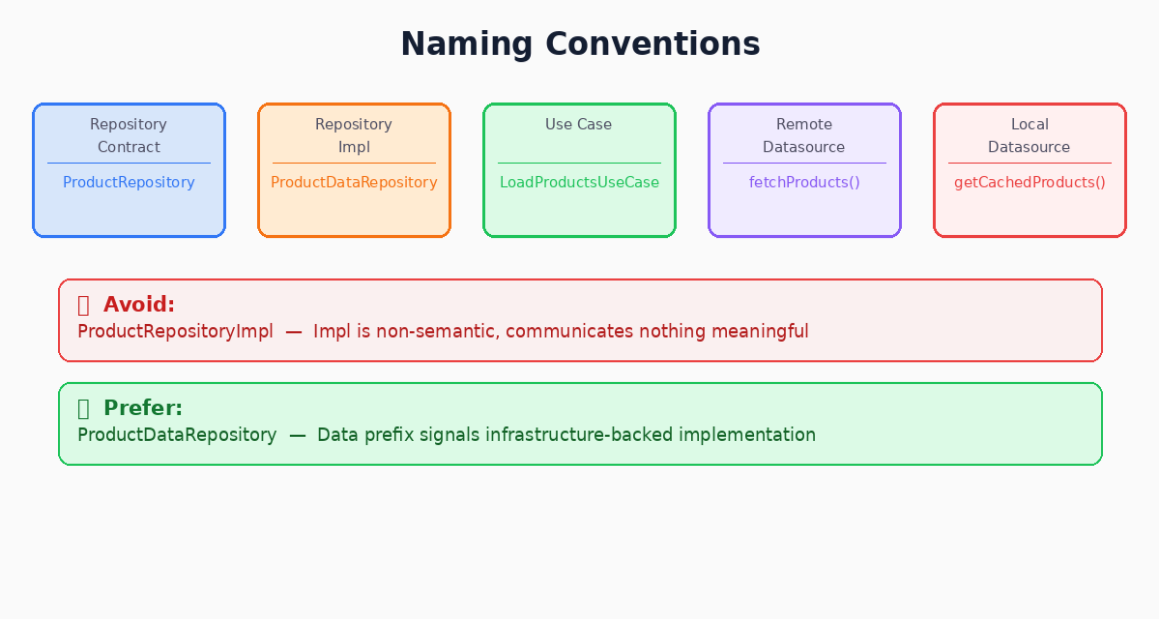


Figure 3 — Naming conventions legend across all architectural layers

Naming Semantics: Load vs Fetch vs Get

Verb	Intent
`load…()`	Source-agnostic retrieval. Repository-backed. Preferred default for repository and use case methods.
`fetch…()`	Explicit remote/network retrieval. Used in remote datasources to signal network I/O.
`get…()`	Synchronous or local/in-memory retrieval. Used in local datasources for cached data.

Repository Naming: Avoid Impl

❌ Avoid: ProductRepositoryImpl
Impl is non-semantic — it communicates nothing about the nature of the implementation.

✅ Prefer: ProductDataRepository
The Data prefix explicitly signals an infrastructure-backed implementation.

6. Data Layer

The data layer handles all infrastructure access: API communication, local persistence, DTO serialisation, caching, and mapping. It contains repository implementations, mappers, and datasources.

Datasource Types

Type	Responsibility
`Remote Datasource`	GraphQL, REST, Firebase — handles raw network I/O, returns DTOs.
`Local Datasource`	SharedPreferences, Hive, SQLite — handles on-device persistence, returns DTOs.
`Cache Datasource`	Cache-first strategies, offline-first support, synchronisation logic.

DTOs

DTOs (Data Transfer Objects) represent transport-layer models and serialised API responses. They must remain inside the data layer and must never leak into domain or presentation.

Mappers

Mappers transform DTOs into domain entities. Recommended approach: one mapper per aggregate or entity, implemented as extension methods in separate mapper files.

7. Shared Folder

The shared folder contains cross-feature abstractions that are not owned by any single feature. It provides base classes, interfaces, and infrastructure helpers that multiple features can depend on without creating direct feature-to-feature coupling.

> **Key Rule**
>
> **Features must never depend on each other directly.**
> When two features share a concept, that concept belongs in shared/, not in either feature.

Figure 4 — Shared folder structure showing base_domain and base_data with their contents

```
lib/
├── features/
│   ├── auth/
│   ├── products/
│   ├── cart/
│   └── …
│
└── shared/
      ├── base_domain/
      │   ├── entities/
      │   ├── repositories/
      │   ├── use_cases/
      │   ├── value_objects/
      │   ├── failures/
      │   └── extensions/
      │
      └── base_data/
      ├── datasources/
      ├── dtos/
      ├── mappers/
      ├── repositories/
      ├── network/
      └── cache/
```

base_domain/

Contains pure Dart abstractions shared across multiple features. Has no framework, API, or infrastructure knowledge. All classes inside base_domain must remain as framework-independent as the domain layer of any individual feature.

Directory	Contents
entities/	Abstract or base entity classes extended by feature entities.
repositories/	Base repository interfaces that feature repository contracts extend.
use_cases/	Base use case interfaces or abstract classes (e.g. UseCase<Type, Params>).
value_objects/	Shared value objects used across multiple domains (e.g. Money, Email, PhoneNumber).
failures/	Shared domain failure types (e.g. NetworkFailure, CacheFailure, ValidationFailure).
extensions/	Pure Dart domain-level extension methods shared across features.

base_domain Rule: No Flutter imports. No data layer imports. No feature imports.
Only pure Dart. base_domain is a dependency of feature domain layers, never the reverse.

base_data/

Contains shared infrastructure implementations. Features extend these base classes in their own data layer rather than duplicating boilerplate. base_data may import base_domain but must never import any feature.

Directory	Contents
`datasources/`	Abstract base datasource classes (e.g. BaseRemoteDatasource, BaseLocalDatasource).
`dtos/`	Shared DTO base classes or mixins (e.g. pagination wrappers, error response models).
`mappers/`	Base mapper interfaces or abstract classes extended by feature mappers.
`repositories/`	Base repository implementation helpers used by feature data repositories.
`network/`	HTTP client setup, interceptors, token refresh logic, connectivity handling.
`cache/`	Cache strategy helpers, expiry logic, key generators used by feature caches.

base_data Rule: May import base_domain. Must NOT import any feature. Must NOT import presentation.
base_data is a dependency of feature data layers, never the reverse.

Dependency Direction with shared/

The shared folder extends the original dependency rule without breaking it. base_domain sits alongside feature domain layers and is treated as a peer dependency. base_data follows the same pattern for the data layer.

```
feature domain/     →  base_domain/  (extends base entities, implements base use
case interfaces)
feature data/       →  base_data/ (extends base datasources, mappers, repository
helpers)
feature data/       →  base_domain/  (same as before: data depends on domain)

base_data/          →  base_domain/  (base data may reference base domain types)
base_domain/        →  nothing    (still depends on nothing)
```

Features must never import each other.
Shared abstractions must live in shared/, not be copied across features.

What Belongs in shared/ vs Inside a Feature

Concept	Location and Reason
`UseCase<Type, Params> base class`	shared/base_domain/use_cases/ — used by every feature
`Failure hierarchy (NetworkFailure…)`	shared/base_domain/failures/ — used by all repository contracts
`Money / Email value objects`	shared/base_domain/value_objects/ — shared business concepts
`HTTP client + interceptors`	shared/base_data/network/ — one implementation, all features
`Pagination DTO wrapper`	shared/base_data/dtos/ — repeated across remote datasources
`ProductEntity`	features/products/domain/entities/ — belongs to products feature only
`CartRepository contract`	features/cart/domain/repositories/ — belongs to cart feature only

8. Testing Structure

Tests mirror the production folder structure exactly, providing discoverability, ownership clarity, and scalability.

```
test/
├── features/
│   └── feature_name/
│       ├── data/
│       │   ├── datasources/
│       │   ├── mappers/
```

```
│       │   └── repositories/
│       ├── domain/
│       │   └── use_cases/
│       └── presentation/
│           ├── pages/
│           └── cubits/
└── shared/
    ├── base_domain/
    └── base_data/
```

9. Architectural Rules Summary

Component	Rule
`Domain`	No Flutter imports. No API knowledge. No DTO knowledge. Pure Dart only.
`Presentation`	No datasource access. No infrastructure logic. Communicates only through use cases or repository contracts.
`Data`	Infrastructure-specific logic only. No UI logic. Returns entities via repositories.
`Repository`	Returns entities only. Hides all infrastructure details. Never exposes transport mechanisms.
`Cubit`	Calls use cases only. Never accesses datasources directly. Exposes immutable state.
`Datasource`	Returns DTOs only. Never exposes entities. Contains infrastructure-specific logic.
`base_domain`	Pure Dart only. No Flutter, no data layer, no feature imports. Extended by feature domain layers.
`base_data`	May import base_domain. Must not import any feature or presentation layer.
`shared/`	Features depend on shared/, never the reverse. No feature-to-feature imports.

Core Principle

Business logic must remain independent from infrastructure and UI implementation details. Domain code should be readable, testable, and reusable without knowledge of how data is fetched, stored, or displayed.